



Mestrado em Engenharia Informática

2025/26

J.L. Sobral

Memory Hierarchy

Parallel speedup



speedup (gain)

- execution time of the best sequential implementation divided by the execution time of the parallel version

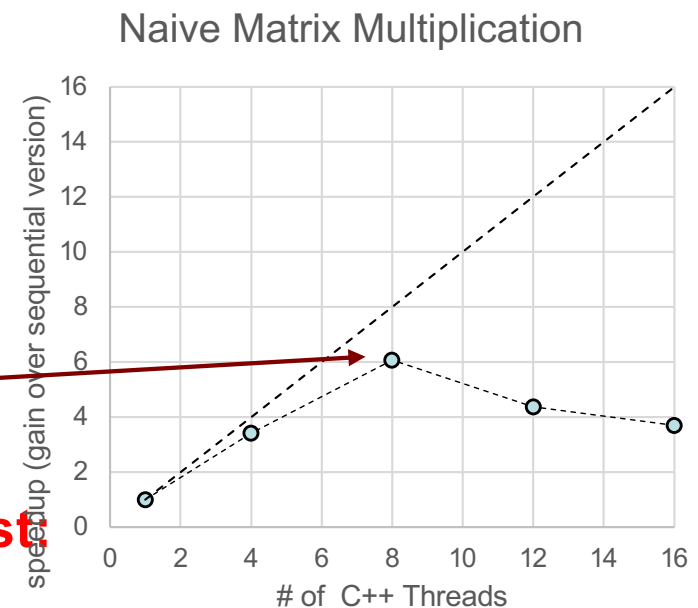
[traditional] scalability analysis

- Analyse the speedup [increase?] as the number of Processing Units (PUs) increases

This week best PL question:

“Is there any automatic way to find the best number of PUs to use?”

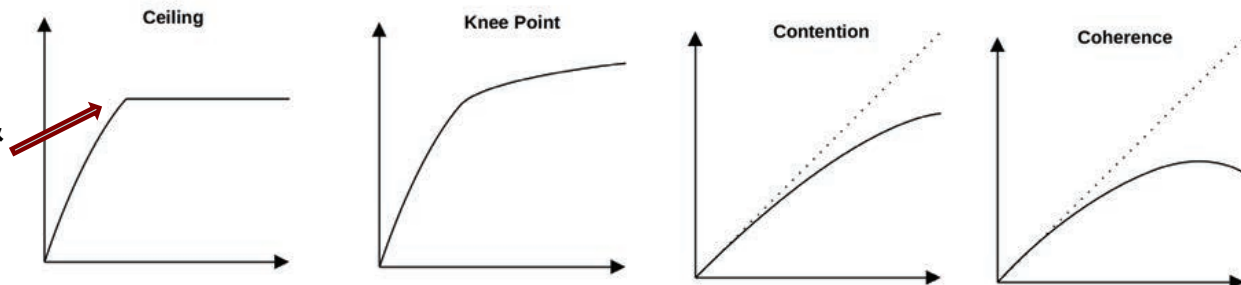
more generic question must be answered first:
“Why aren’t we getting ideal speedup?”



The scalability profile helps to identify the type of limitation:

- critical path (& *serial fraction* & schedule & #PUs) => ceiling

- Stay tuned for more...



Principles of Computer Design

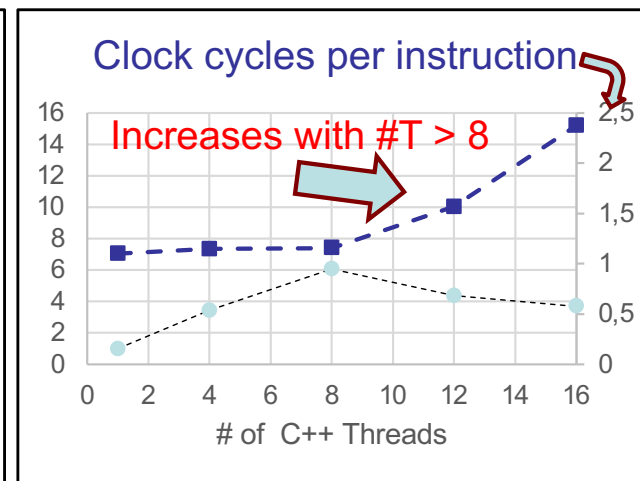
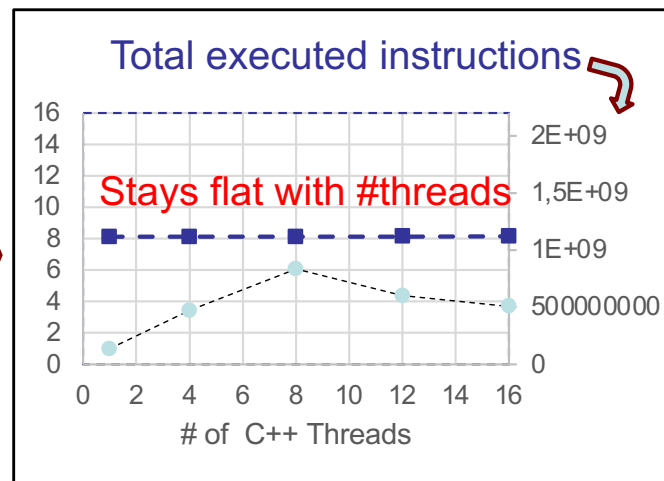


• The Processor Performance Equation & multicore

CPU time = Instruction count × Cycles per instruction × Clock cycle time

$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

The processor performance equation provides one **first insight** on the lack of scalability of our naïve MM implementation



Now we should further investigate this CPI behaviour

Peak FLoating-Point OPerations per Second (FLOPs) = 1 x 1 x 2 x 2.9 GHz = 6 GFlops
(#PUs x FP per instruction x Instructions per cycle x Cycles per second)

Real performance = #FLOP_in_our_code / Texe = 2 x 512³ / 0.45 = 0.6 GFlops (10%)

second insight: base performance is low & drops when #PUs > 8

Memory hierarchy: locality



- **Locality principle**

"Programs tend to access a limited portion of memory in a given period of time"

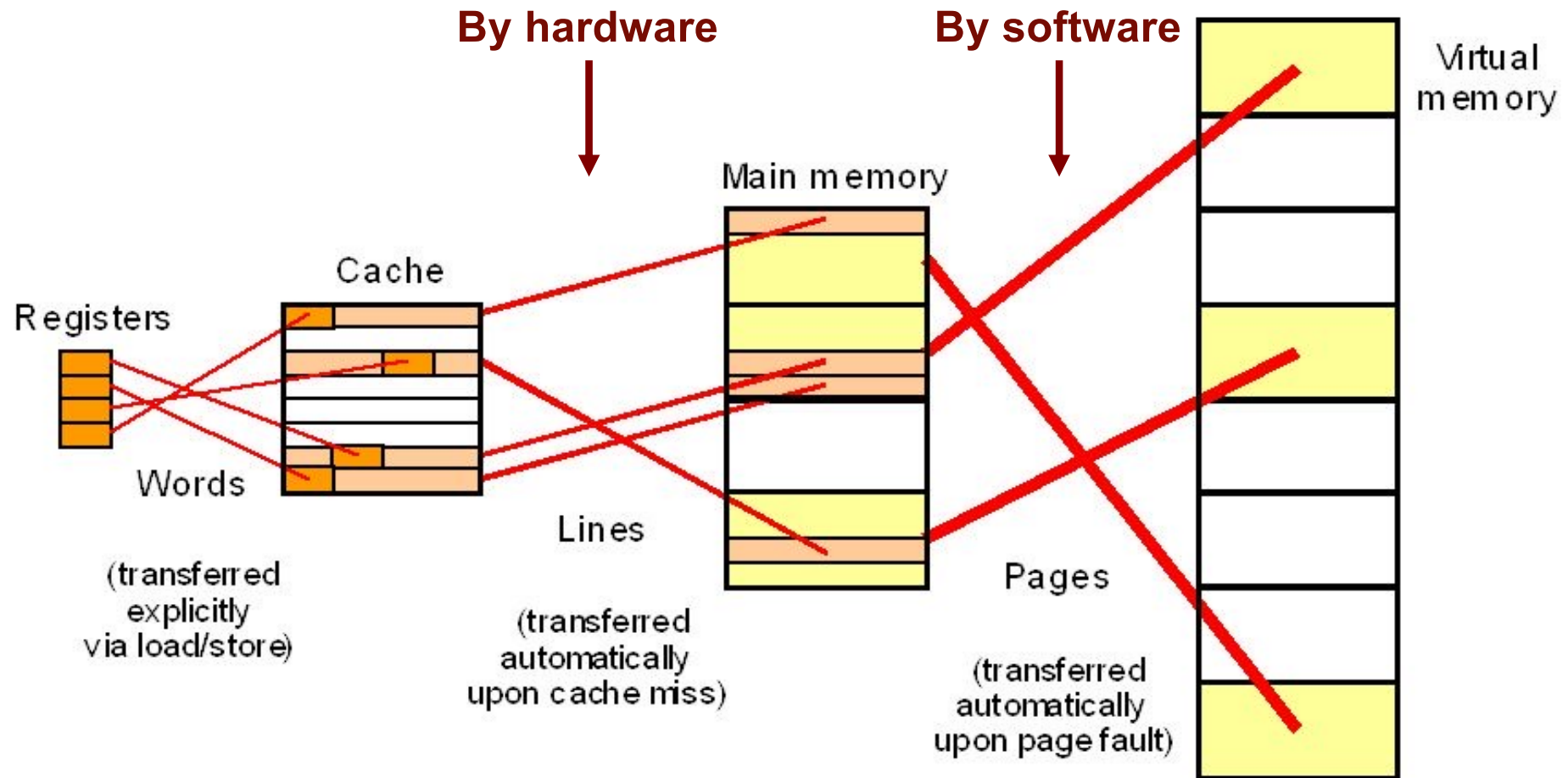
- allows faster *[local]* memory to be used to store the most frequently/recently used information
- takes advantage of **memory bandwidth**, since the information transferred between different levels of the hierarchy can be done in blocks (& pre-fetched)
- **Spatial** locality
 - if a memory element is accessed by the PU, then elements with nearby addresses will most likely be accessed in the near future
- **Temporal** locality
 - A memory element accessed by the PU will most likely be accessed again in the near future

- Examples

- **Spatial**: n consecutive accesses to `a`
- **Temporal**: n accesses to `sum`

```
sum = 0;
for (i = 0; i < n; i++)
    sum += a[i];
return sum;
```

Memory hierarchy: the big picture



Data movement in a memory hierarchy.

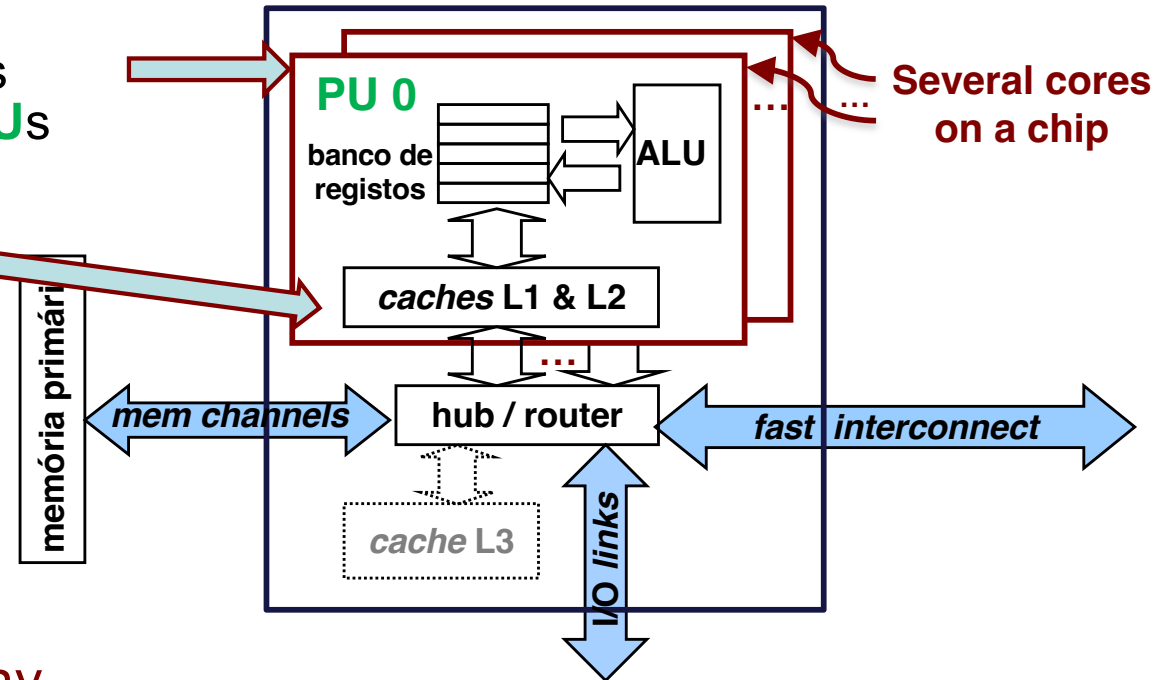
Cache memory & multicore systems



Multicores: peak performance is proportional to the number of **PU**s

Each **PU** owns an exclusive **cache memory** L1, L2 ...

- Transparently takes advantage of **spatial** and **temporal** locality
- Data and instruction L1 caches are normally exclusive to PUs
- L2, L3, ...caches can be shared by several cores



Sharing memory among many PUs brings additional problems:

- **Coherence** of the cache information

Data might be replicated on several PUs

Writes to a local cache must invalidate all copies of the same data on other PUs caches

- **Sharing** of memory & network

Memory bandwidth is shared by all PUs (note: L1 bandwidth scales with #PUs)

Arquiteturas multicore e memória cache

Exemplo: raptor core)



Raptor Cove (2022-Gen13)

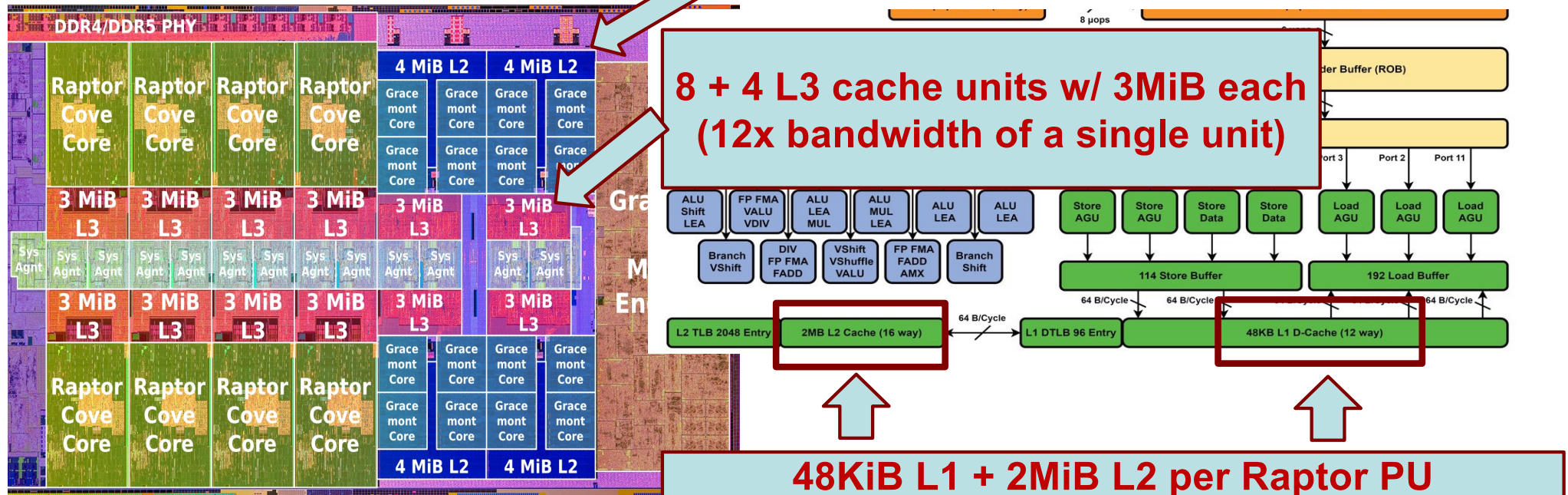
6-vias / 12-vias (Intel: ~20% less CPI)

Intel i9 13900K (custa cerca de 600€)

- 8 Raptor Cove cores (3.0 a 5.8 GHz)
- 16 Gracemount cores (2.2 a 4.3 GHz)

**8 + 16
PUs**

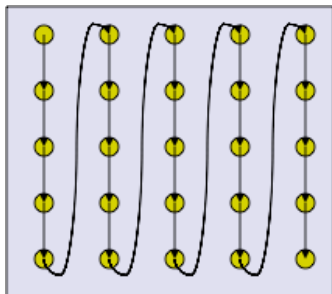
**4 MiB L2 partilhada
por 4 Gracemount PUs**



Making Code "Friendly" of the Memory Hierarchy



- How to make the implementation more cache-friendly?
 - Minimize the number of blocks transferred between the various memory levels (i.e., the number of cache misses)
- Code/data reorganization to improve **spatial locality**
 - Reorganization of code blocks
 - Arrange blocks in memory in the order in which they are executed
 - Reorganize data access
 - Access data elements in the order they are stored



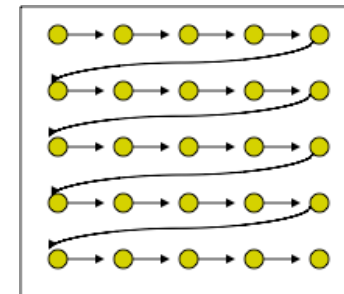
Transfers 5000*100 Blocks

Before

```
for (j = 0; j < 100; j = j+1)
  for (i = 0; i < 5000; i = i+1)
    x[i][j] = 2 * x[i][j];
```

After

```
for (i = 0; i < 5000; i = i+1)
  for (j = 0; j < 100; j = j+1)
    x[i][j] = 2 * x[i][j];
```



Transfers 5000*100/B blocks

Making Code "Friendly" of the Memory Hierarchy



Process the data by blocks (which fit in the cache) to improve the spatial/temporal locality in accessing the data:

Accessing/storing data in blocks, example: transposing an array

Before	After
<pre>for (i = 0; i < N; i = i+1) for (j = 0; j < N; j = j+1) dst[i][j] = src[j][i];</pre>	<pre>for (int ii = 0; ii < N; ii += blocksize) for (int jj = 0; jj < N; jj += blocksize) // transpose the block beginning at [ii,jj] for (int i = ii; i < ii + blocksize; i++) for (int j = jj; j < jj + blocksize; j++) dst[i][j] = src[j][i];</pre>

src

1	1						
2	2						
3	3						
4	4						
5	5						
6	6						
7	7						
8	8						

dst

1	2	3	4	5	6	7	8
1	2	3	4	5	6	7	8

src

1	1	1	1				
2	2	2	2				
3	3	3	3				
4	4	4	4				

dst

1	2	3	4				
1	2	3	4				
1	2	3	4				
1	2	3	4				